KodeKloud

Container Orchestration Storage › **Container Storage Interface**

Container Orchestration Storage

Container Storage Interface

Copy page

The article explains the Container Storage Interface and its role in standardizing storage driver interactions with container orchestration platforms.

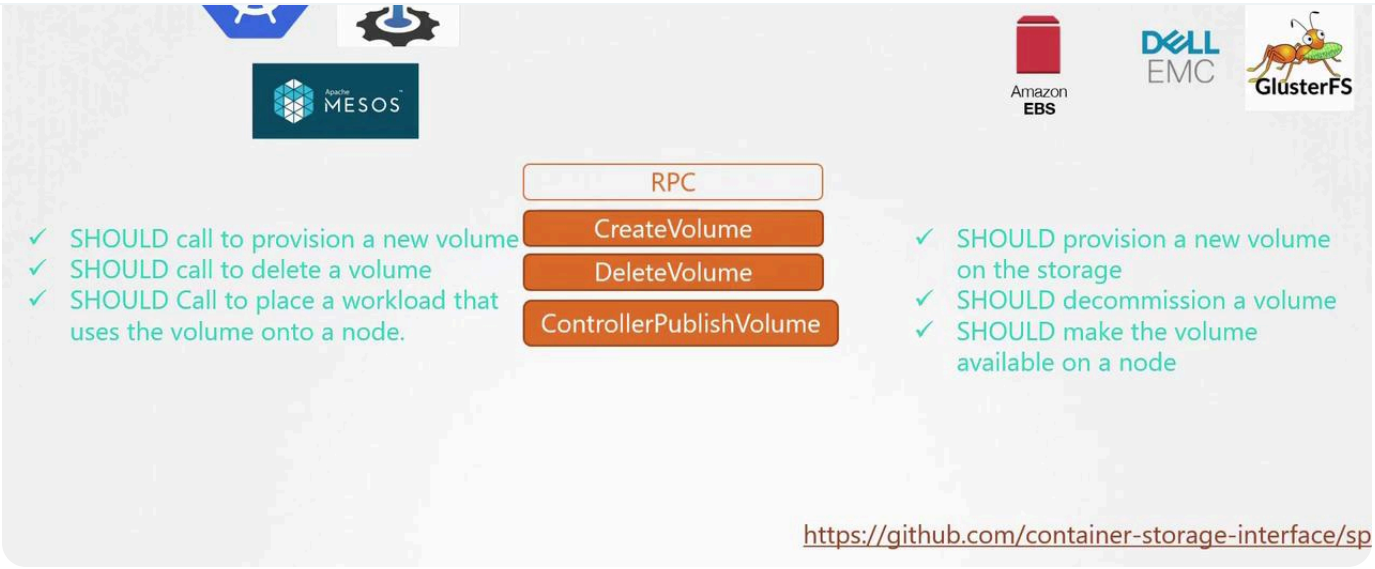
The Container Storage Interface (CSI) plays a crucial role in modern container orchestration by standardizing how storage drivers interact with container platforms like Kubernetes, Cloud Foundry, and Mesos. In earlier container environments, Kubernetes integrated container runtime functionality directly into its source code using Docker. However, with the advent of alternative container runtimes—including Rocket and CRI—it became essential to decouple container runtime operations from Kubernetes. This led to the introduction of the Container Runtime Interface (CRI), and similarly, the need for modularity extended to networking and storage with the Container Networking Interface (CNI) and Container Storage Interface (CSI), respectively.

Why CSI Matters

CSI was designed to support a wide array of storage solutions without being bound to Kubernetes-specific implementations. By leveraging CSI, developers can create custom drivers for diverse storage systems, thereby promoting flexibility and innovation. Major storage vendors, including Portworx, Amazon EBS, Azure Disk, Dell EMC, Isilon, PowerMax, Unity, XtremIO, NetApp, Nutanix, HPE, Hitachi, and Pure Storage, have adopted this standard. The vendor-agnostic nature of CSI ensures that any compliant container orchestration tool can interface seamlessly with any CSI-supported storage system.

The diagram below encapsulates the CSI architecture, detailing logos, Remote Procedure Call (RPC) workflows, and guidelines for managing volumes in containerized environments:

KodeKloud



How CSI Works

At its essence, CSI defines a series of RPCs used by container orchestrators to interact with storage drivers. Here's an overview of the typical interactions:

Volume Creation: When a pod is deployed and requires a storage volume, Kubernetes initiates a "Create Volume" RPC call, providing details such as the volume name and configuration parameters. The corresponding storage driver then provisions a new volume on the storage array and returns the operation status.

Volume Deletion: Conversely, when a volume is no longer needed, Kubernetes makes a "Delete Volume" RPC call. The storage driver processes the decommissioning of that specific volume and communicates the result back to Kubernetes.

Each RPC defined by CSI includes detailed specifications regarding the parameters sent by the caller, the expected responses from the storage solution, and the error codes to be used in various scenarios.

For an in-depth look at the CSI specifications, visit the full **CSI specification on GitHub**.

KodeKloud

across various storage systems and container runtimes, fostering a more flexible and dynamic container ecosystem. Stay tuned for our next article where we will dive deeper into related topics in container storage and management.

Watch Video

Learn more

Was this page helpful? Yes No

Volume Driver Plugins in Docker
Previous

Volumes
Next

Powered by mintlify